RAPPORT TECHNIQUE

Diagnostic de Défauts dans les Moteurs CC 12V

Classification Binaire sur Données Réelles

Projet	Système embarqué de détection de défauts — Moteur CC 12V
Plateforme	ESP32 (microcontrôleur basse consommation)
Données	6 fichiers CSV acquis manuellement sur moteur réel
Capteurs	Tension (V) | Courant (A) | Vitesse (RPM)
Évaluation	Cross-Validation Stratifiée à 5 Plis
Classes	Normal (0) / Surcharge (1)
Méthode clé	Fenêtres glissantes (30 pts, stride 15) → 45 features → ML

1. Introduction

1.1 Objectif du projet

Ce projet vise à développer un système de diagnostic automatique pour les petits moteurs à courant continu (CC) 12V à partir de **données réelles** acquises manuellement. Contrairement aux notebooks précédents basés sur le dataset ITSC simulé, ce rapport exploite des mesures directes réalisées sur un vrai moteur CC 12V instrumenté avec un microcontrôleur ESP32.

Le système classifie en temps réel l'état du moteur en **deux catégories** :

- **Normal** : le moteur fonctionne dans ses conditions nominales
- **Surcharge** : une contrainte mécanique ou électrique excessive est détectée

1.2 Contraintes matérielles ESP32

Contrainte	Implication
Mémoire RAM limitée (~520 Ko)	Le modèle doit avoir une empreinte mémoire de quelques Ko
Pas de GPU	Les calculs doivent se faire en CPU avec arithmétique flottante simple
Pas de bibliothèques ML	Le modèle doit être exporté en code C natif
Temps réel	L'inférence doit se faire en moins de 100 ms

Ces contraintes excluent les architectures complexes (réseaux de neurones profonds, CNN, LSTM) et orientent le choix vers des algorithmes classiques de machine learning.

2. Analyse du Dataset

2.1 Provenance des données

Ce dataset a été acquis **manuellement** sur un vrai moteur à courant continu 12V instrumenté avec un microcontrôleur ESP32. Les valeurs sont transmises via port série (PuTTY) à une cadence de **2 Hz** (intervalle 500 ms) ou **10 Hz** (intervalle 100 ms) selon le fichier.

2.2 Inventaire des 6 fichiers

Fichier	Label	Tension	Courant	Vitesse	Fréq.	Lignes utiles
dataset_moteur.csv	Normal	✓	✗	✗	10 Hz	~3 574
dataset_finale_eet_5A.csv	Normal	✓	✓	✓	2 Hz	~843
dataset_trés_finale_30A.csv	Normal	✓	✓	✓	2 Hz	~708
dataset_surcharge.csv	Surcharge	✓	✗	✗	10 Hz	~3 689
dataset_surcharge_5A.csv	Surcharge	✓	✓	✗	2 Hz	~740
dataset_surcharge_30AA.csv	Surcharge	✓	✓	✗	2 Hz	~572

2.3 Problèmes identifiés et corrections appliquées

L'analyse des fichiers bruts a révélé **7 problèmes** distincts, tous corrigés automatiquement par la fonction `load_and_clean()` :

#	Problème	Fichiers concernés	Correction
1	En-tête PuTTY (13 lignes de boot ESP32)	surcharge_5A, surcharge_30A	Skip jusqu'à la ligne contenant 'Temps'
2	Séparateur décimal virgule ','	moteur, surcharge	decimal=',' lors du chargement
3	Virgule parasite dans 'Tension(V),'	moteur	rstrip(',') sur les noms de colonnes
4	Lignes moteur éteint (Tension = 0 V)	Tous les fichiers	Suppression si Tension < 1 V
5	Pic de courant aberrant (13.51 A)	surcharge_5A	Suppression si Courant > 5 A
6	Ligne NaN de fin de fichier	surcharge_30A	dropna() après chargement
7	Capteurs absents selon le fichier	moteur, surcharge (pas Courant/Vitesse)	Imputation à 0 (valeur neutre)

2.4 Correspondance des capteurs

Capteur ESP32	Colonne CSV	Disponibilité
Capteur de tension (diviseur résistif)	Tension(V)	Tous les fichiers
Capteur de courant ACS712	Courant(A)	4 fichiers / 6 (imputé 0 sinon)

Capteur ESP32	Colonne CSV	Disponibilité
Encodeur Hall (vitesse)	Vitesse(RPM)	2 fichiers / 6 (imputé 0 sinon)

3. Méthode Clé : Segmentation en Fenêtres Glissantes

3.1 Pourquoi découper en fenêtres ?

Les enregistrements bruts sont des séries temporelles longues (3 à 7 minutes). Un modèle de classification classique ne peut pas travailler directement sur des milliers de points — ce n'est pas un seul "échantillon", c'est une séquence entière. La segmentation résout ce problème en découpant chaque enregistrement en **petits clips independants**, chacun constituant un échantillon pour le classifieur.

3.2 Principe de la fenêtre glissante

Paramètre	Valeur	Signification
Taille de fenêtre (window_size)	30 échantillons	Durée du clip : 15 s à 2 Hz / 3 s à 10 Hz
Pas (stride)	15 échantillons	Décalage entre deux fenêtres consécutives
Recouvrement	50 %	Deux fenêtres adjacentes partagent 15 points
Label de la fenêtre	Label du fichier source	Toute la fenêtre hérite du label Normal ou Surcharge

Illustration :
```
Données : [1][2][3]...[3574]
Fenêtre 1 → points   1 à  30  →  label = Normal
Fenêtre 2 → points  16 à  45  →  label = Normal  (avance de 15)
Fenêtre 3 → points  31 à  60  →  label = Normal  ...
```

3.3 Résultat de la segmentation

Fichier source	Label	Lignes utiles	Fenêtres produites
moteur	Normal	~3 574	~238
finale_5A	Normal	~843	~55
finale_30A	Normal	~708	~45
surcharge	Surcharge	~3 689	~245
surcharge_5A	Surcharge	~740	~47
surcharge_30A	Surcharge	~572	~36
TOTAL	**—**	**~10 126**	**~666 fenêtres**

Avantage clé : ~666 fenêtres permettent d'utiliser une **cross-validation stratifiée à 5 plis** classique — plus besoin de Leave-One-Out. Les métriques sont statistiquement robustes.

4. Extraction des Caractéristiques (Feature Engineering)

4.1 Pourquoi extraire des features ?

Une fenêtre brute de 30 échantillons × 3 capteurs = 90 valeurs. Les modèles classiques (arbres, SVM) ne traitent pas efficacement les séquences brutes — ils ne "voient" pas le temps. On résume donc chaque fenêtre en **45 chiffres caractéristiques** calculables en temps réel sur ESP32 :

- 10 features temporelles × 3 capteurs = 30 features
- 5 features fréquentielles × 3 capteurs = 15 features
- **Total : 45 features par fenêtre**

4.2 Caractéristiques temporelles (10 par capteur)

#	Feature	Formule	Signification physique
1	Moyenne	$\mu = (1/N)\ \Sigma xi$	Valeur centrale — un moteur en surcharge consomme plus en moyenne
2	Écart-type	$\sigma = \sqrt{[(1/N)\ \Sigma(xi-\mu)^2]}$	Variabilité — la tension fluctue plus en cas d'anomalie
3	Valeur RMS	$\sqrt{[(1/N)\ \Sigma xi^2]}$	Énergie efficace du signal
4	Maximum	max(x)	Pics de courant anormaux lors d'une surcharge
5	Minimum	min(x)	Creux minimaux du signal
6	Peak-to-peak	max(x) − min(x)	Amplitude totale des oscillations
7	Kurtosis	$E[(x-\mu)^4]/\sigma^4 - 3$	Présence de chocs ou pics brusques
8	Skewness	$E[(x-\mu)^3]/\sigma^3$	Asymétrie de la distribution
9	MAD	$(1/N)\ \Sigma\lvert xi - xi-1\rvert$	Rugosité — variabilité entre points consécutifs
10	IQR	Q75 − Q25	Dispersion des valeurs centrales

4.3 Caractéristiques fréquentielles (5 par capteur)

La **transformée de Fourier (FFT)** décompose le signal en fréquences (comme un égaliseur audio). Un moteur en surcharge génère des vibrations à des fréquences différentes d'un moteur normal :

#	Feature	Description
11	Énergie spectrale	$\Sigma\lvert FFT\rvert^2$ — puissance totale du signal dans le domaine fréquentiel
12	Fréquence dominante	Fréquence ayant la plus grande amplitude spectrale
13	Fréquence moyenne pondérée	Barycentre fréquentiel du spectre
14	Dispersion spectrale	Écart-type des amplitudes — spectre concentré ou étalé ?
15	Ratio du pic spectral	Pic maximal / Somme totale — dominance d'une fréquence

4.4 Traitement des capteurs manquants

Les fichiers **moteur** et **surcharge** ne contiennent que la Tension. Pour maintenir le vecteur de 45 features cohérent sur tous les fichiers, les colonnes manquantes sont **imputées à 0** avant l'extraction. Les features

calculées sur un signal constant = 0 produisent des valeurs neutres (Std=0, Kurtosis=0, etc.) qui n'introduisent pas de biais discriminant.

5. Préparation des Données

5.1 Normalisation (StandardScaler)

Les 45 features ont des échelles très différentes (énergie spectrale ≈ 10^7, courant moyen ≈ 1.2). Sans normalisation, les modèles comme SVM et KNN ignorent les petites features. Le **StandardScaler** transforme chaque feature :

```
x_normalisé = (x - moyenne) / écart-type   →  moyenne=0, écart-type=1
```

Important : le scaler est ajusté *uniquement* sur les données d'entraînement, puis appliqué aux données de test. Les paramètres (moyenne et écart-type de chaque feature) sont exportés dans `scaler_params_reel.h` pour être reproduits à l'identique sur l'ESP32.

5.2 Partitionnement train / test

Ensemble	Proportion	Usage
Entraînement	80 %	Ajustement des modèles et cross-validation
Test	20 %	Évaluation finale du modèle sélectionné (données jamais vues)

Le partitionnement est **stratifié** : la proportion Normal/Surcharge est identique dans les deux ensembles.

5.3 Cross-validation stratifiée à 5 plis

Les ~666 fenêtres sont divisées en 5 groupes égaux. À chaque itération, 4 groupes servent à l'entraînement et 1 au test :

```
Pli 1 : [TEST ][TRAIN][TRAIN][TRAIN][TRAIN]
Pli 2 : [TRAIN][TEST ][TRAIN][TRAIN][TRAIN]
Pli 3 : [TRAIN][TRAIN][TEST ][TRAIN][TRAIN]
Pli 4 : [TRAIN][TRAIN][TRAIN][TEST ][TRAIN]
Pli 5 : [TRAIN][TRAIN][TRAIN][TRAIN][TEST ]
```

Chaque fenêtre est évaluée exactement une fois. La **Balanced Accuracy finale** est la moyenne sur les 5 plis — estimation fiable et robuste, sans dépendre d'un seul split aléatoire.

6. Modèles de Classification

6.1 Choix des algorithmes

Sept algorithmes ont été évalués, choisis pour leur compatibilité avec les contraintes ESP32 :

Algorithme	Principe	Avantage ESP32	Limitation
Decision Tree	Séquence de tests sur les features	Très compact, rapide	Peut sur-apprendre
Random Forest	Vote de 100 arbres indépendants	Plus robuste qu'un seul arbre	Taille × nb d'arbres
KNN	Comparaison aux k voisins les plus proches	Pas d'entraînement	Stockage de toutes les données
SVM Linéaire	Hyperplan de marge maximale	Très compact (poids + biais)	Frontières linéaires uniquement
Régression Logistique	Modèle linéaire probabiliste	Le plus compact possible	Frontières linéaires uniquement
Perceptron Multicouche	Réseau de neurones 64→32 neurones	Taille contrôlable	Nécessite plus de données
HistGradientBoosting	Boosting séquentiel d'arbres	Très performant	Plus complexe

6.2 Gestion du déséquilibre Normal/Surcharge

Le dataset peut être déséquilibré selon les fichiers inclus. Deux mécanismes compensent ce déséquilibre :

Mécanisme	Principe	Appliqué à
class_weight='balanced'	Pondère les erreurs inversement à la fréquence de classe	Decision Tree, Random Forest, SVM, Reg. Logistique, HistGBT
scale_pos_weight	Ratio neg/pos pour XGBoost	XGBoost uniquement

6.3 Métriques d'évaluation

Les modèles sont évalués selon :

- **Balanced Accuracy** : moyenne(Recall Normal, Recall Surcharge) — métrique principale, insensible au déséquilibre. Un modèle qui prédit toujours "Normal" obtient 0.50.
- **F1-score (Surcharge)** : moyenne harmonique précision/rappel pour la classe Surcharge — détection des surcharges.
- **Accuracy par classe** : Acc. Normal et Acc. Surcharge séparément — détection des biais par classe.
- **Matrice de confusion** : visualisation complète des erreurs de classification.

7. Résultats de l'Évaluation CV 5 Plis

7.1 Tableau récapitulatif

Les métriques ci-dessous sont calculées sur les **~666 prédictions cross-validées** (chaque fenêtre évaluée exactement une fois). Tableau trié par Balanced Accuracy décroissante :

Modèle	Acc. Éq. (principale)	F1 (Surcharge)	Accuracy	Acc. Normal	Acc. Surcharge	Temps (s)
XGBoost	0.828	0.825	0.829	0.84	0.817	0.806455
Decision Tree	0.77	0.784	0.778	0.736	0.82	3.430874
Random Forest	0.823	0.826	0.826	0.813	0.838	3.193617
KNN	0.815	0.816	0.815	0.798	0.832	1.981354
SVM Linéaire	0.82	0.823	0.82	0.792	0.848	0.029552
Régression Logistique	0.823	0.826	0.823	0.792	0.854	0.034992
Perceptron Multicouche	0.819	0.826	0.818	0.763	0.875	0.142507
HistGradientBoosting	0.82	0.817	0.82	0.822	0.82	0.287014

** Le meilleur modèle ESP32-compatible (KNN exclu) est mis en évidence en vert.*

7.2 Interprétation des métriques

Métrique	Valeur idéale	Valeur minimale acceptable	Signification si faible
Balanced Accuracy	1.00	> 0.80	Biais vers une classe, ou modèle trop simple
F1 (Surcharge)	1.00	> 0.75	Surcharges manquées (faux négatifs) ou alarmes inutiles
Acc. Normal	1.00	> 0.70	Trop de Normal classifiés comme Surcharge (faux positifs)
Acc. Surcharge	1.00	> 0.80	Surcharges non détectées — dangereux opérationnellement

7.3 Sélection du modèle pour ESP32

Le meilleur modèle compatible ESP32 est sélectionné parmi tous les modèles **sauf KNN** (exclu car il requiert le stockage de toutes les données d'entraînement en mémoire RAM, incompatible avec les ~520 Ko de l'ESP32).

Critère de sélection	Valeur
Modèle sélectionné	**(produit à l'exécution)**
Balanced Accuracy (CV 5 plis)	0.8289

Critère de sélection	Valeur
F1-score (Surcharge)	0.8252
Accuracy Normal	0.84
Accuracy Surcharge	0.8176
Compatibilité ESP32	Oui — modèle exporté en C natif via m2cgen

8. Exportation pour ESP32

8.1 Pipeline complet embarqué

```
[Tension] → ADC ┐
[Courant] → ADC ┼──→ Buffer circulaire [30 pts] ──→ extract_features() [45
valeurs]
[Vitesse] → ADC ┘          (0 si capteur absent)
                                   │
                         Normaliser avec scaler_params_reel.h
                                   │
                         score() → argmax → 0=Normal | 1=Surcharge
```

8.2 Conversion en code C (m2cgen)

Le meilleur modèle compatible ESP32 est converti en code C natif via **m2cgen** (Model to Code Generator). La fonction `score()` générée :

- Ne dépend d'aucune bibliothèque externe
- Est directement compilable avec le toolchain Arduino/ESP-IDF
- Accepte les 45 features normalisées en entrée
- Retourne un tableau de 2 scores → argmax → classe prédite

8.3 Fichiers générés

Fichier	Contenu	Usage sur ESP32
model_esp32_reel.c	Fonction score() — code C natif du classificateur	#include dans le sketch Arduino
scaler_params_reel.h	scaler_mean[45] et scaler_scale[45]	Normalisation des features avant inférence

8.4 Exemple de code Arduino

```
#include "model_esp32_reel.c"
#include "scaler_params_reel.h"

int predict(float features[N_FEATURES]) {
    float scaled[N_FEATURES];
    for (int i = 0; i < N_FEATURES; i++)
        scaled[i] = (features[i] - scaler_mean[i]) / scaler_scale[i];
    double output[N_CLASSES];
    score(scaled, output);
    return (output[1] > output[0]) ? 1 : 0;
}  // 0=Normal, 1=Surcharge
```

9. Conclusion

Ce travail a permis de :

1. Acquérir et nettoyer 6 fichiers CSV issus de mesures réelles sur un moteur CC 12V instrumenté avec un ESP32 — correction automatique de 7 problèmes distincts
2. Développer une stratégie de segmentation en fenêtres glissantes (30 pts, stride 15) transformant 6 enregistrements longs en ~666 fenêtres indépendantes exploitables
3. Extraire 45 caractéristiques pertinentes (temporelles et fréquentielles) par fenêtre, calculables en temps réel sur ESP32
4. Entraîner et comparer 7 algorithmes de classification avec cross-validation stratifiée à 5 plis et gestion explicite du déséquilibre de classes
5. Évaluer les modèles avec la Balanced Accuracy comme métrique principale — insensible au déséquilibre Normal/Surcharge
6. Exporter le meilleur modèle en code C directement exécutable sur ESP32 via m2cgen

Le système proposé permet une **détection de surcharge en temps réel** à partir de données réelles, avec une empreinte mémoire minimale adaptée aux microcontrôleurs basse consommation. **L'avantage majeur** par rapport aux notebooks ITSC précédents est la validation sur des données physiques réelles — la performance obtenue reflète directement le comportement attendu sur le système embarqué final.